AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 01.12.2026

Lab 06

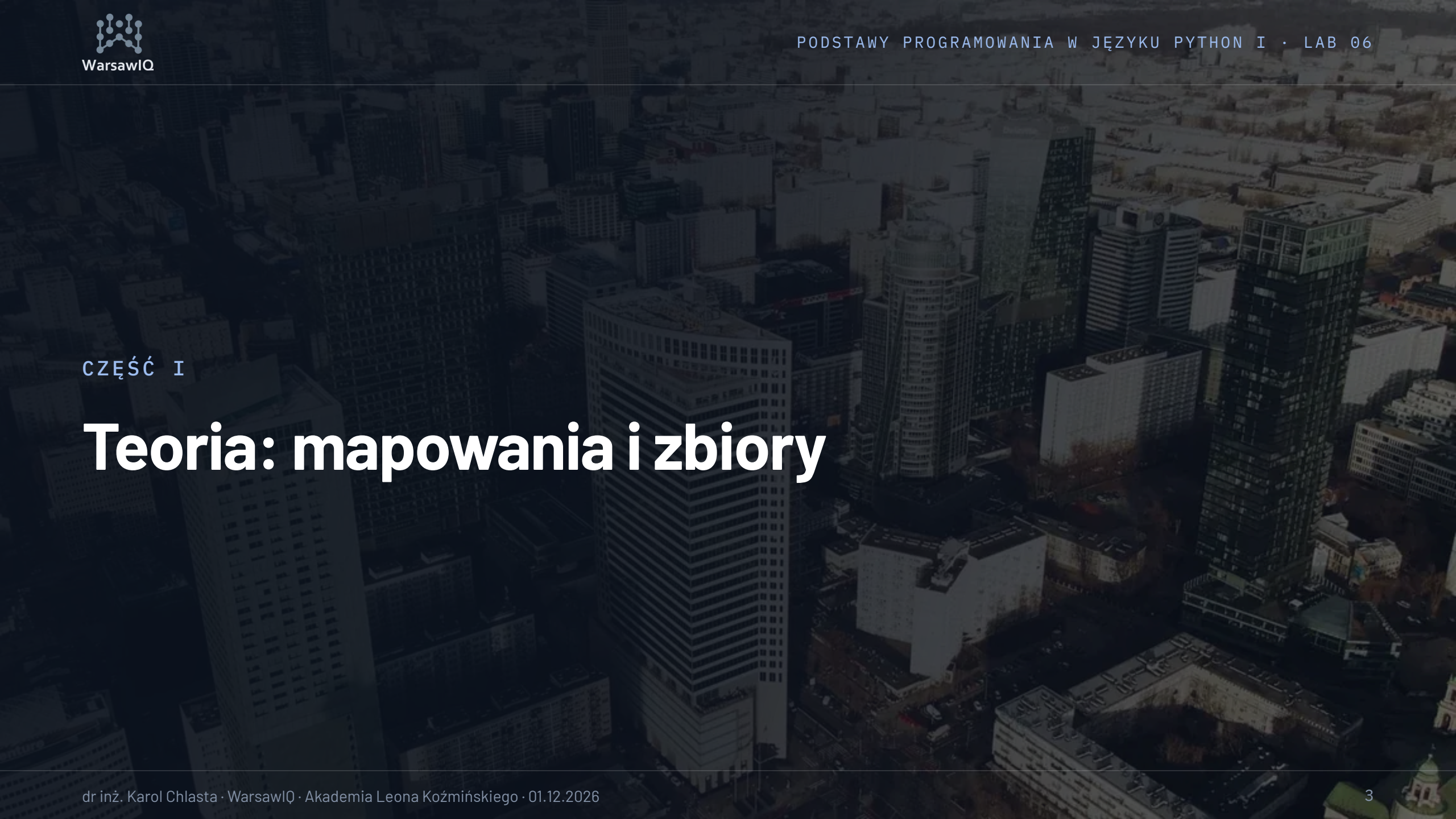
Struktury danych II: słowniki i zbiory

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć **tablicę mieszającą** – mechanizm, dzięki któremu słownik i zbiór są szybkie
- Słowniki: tworzenie, metody `get`, `keys`, `values`, `items`
- Zbiory: suma, część wspólna, różnica
- Mapowanie i filtrowanie danych: książka adresowa, duplikaty, szyfr, częstość słów
- Efekty z sylabusa: **U1**, **U3**, **KS1**



CZĘŚĆ I

Teoria: mapowania i zbiory

Słownik: klucz → wartość

```
phone = {"Ola": "600100200", "Adam": "500600700"}
phone["Ewa"] = "700800900"     # dodanie / nadpisanie
phone["Ola"]                   # '600100200'
phone["Zenon"]                 # KeyError: 'Zenon'
phone.get("Zenon")             # None – bez wyjątku
phone.get("Zenon", "brak")     # 'brak'
del phone["Adam"]
"Ola" in phone                 # True – sprawdza KLUCZE
```

- Klucze są **unikalne**; ponowne przypisanie nadpisuje wartość
- Od Pythona 3.7 słownik **zachowuje kolejność wstawiania** (gwarancja języka)

Metody słownika

WYRAŻENIE	WYNIK
`d.keys()`	widok kluczy
`d.values()`	widok wartości
`d.items()`	widok par `(klucz, wartość)`
`d.get(k, domyślna)`	wartość albo domyślna
`d.pop(k)`	usuwa i zwraca wartość
`d.update(inny)`	dopisuje / nadpisuje pary
`len(d)`	liczba par

```
for name, number in sorted(phone.items()):
    print(f"{name}: {number}")
```

Jak to działa: tablica mieszająca

1. `hash(klucz)` zamienia klucz na liczbę całkowitą
2. Z tej liczby wyliczany jest **numer szufladki** w tablicy
3. Para klucz–wartość trafia do tej szufladki

OPERACJA	LISTA	SŁOWNIK / ZBIÓR
`x in …`	O(n) – szuka po kolei	**O(1)** średnio
dostęp do elementu	po indeksie	po kluczu

- Klucz musi być **hashowalny**: `str`, `int`, `tuple` – tak; `list`, `dict` – nie (są zmienne)
- Od Pythona 3.3 `hash()` tekstów jest **losowany przy każdym uruchomieniu** – ochrona przed atakami DoS, w których napastnik zasypuje serwer kolidującymi kluczami

Zbiory

```
a = {"Ala", "Ola", "Jan"}
b = {"Jan", "Piotr"}
empty = set()          # {} to pusty SŁOWNIK, nie zbiór!
```

OPERACJA	OPERATOR	METODA	WYNIK
suma	`a | b`	`a.union(b)`	`{'Ala', 'Ola', 'Jan', 'Piotr'}`
część wspólna	`a & b`	`a.intersection(b)`	`{'Jan'}`
różnica	`a - b`	`a.difference(b)`	`{'Ala', 'Ola'}`
różnica symetryczna	`a ^ b`	`a.symmetric_difference(b)`	`{'Ala', 'Ola', 'Piotr'}`

- Zbiór **nie ma kolejności** ani duplikatów; do wypisania użyj `sorted()`

Wzorzec: zliczanie słownikiem

```
counts = {}
for word in "to jest to co to jest".split():
    counts[word] = counts.get(word, 0) + 1
# {'to': 3, 'jest': 2, 'co': 1}
```

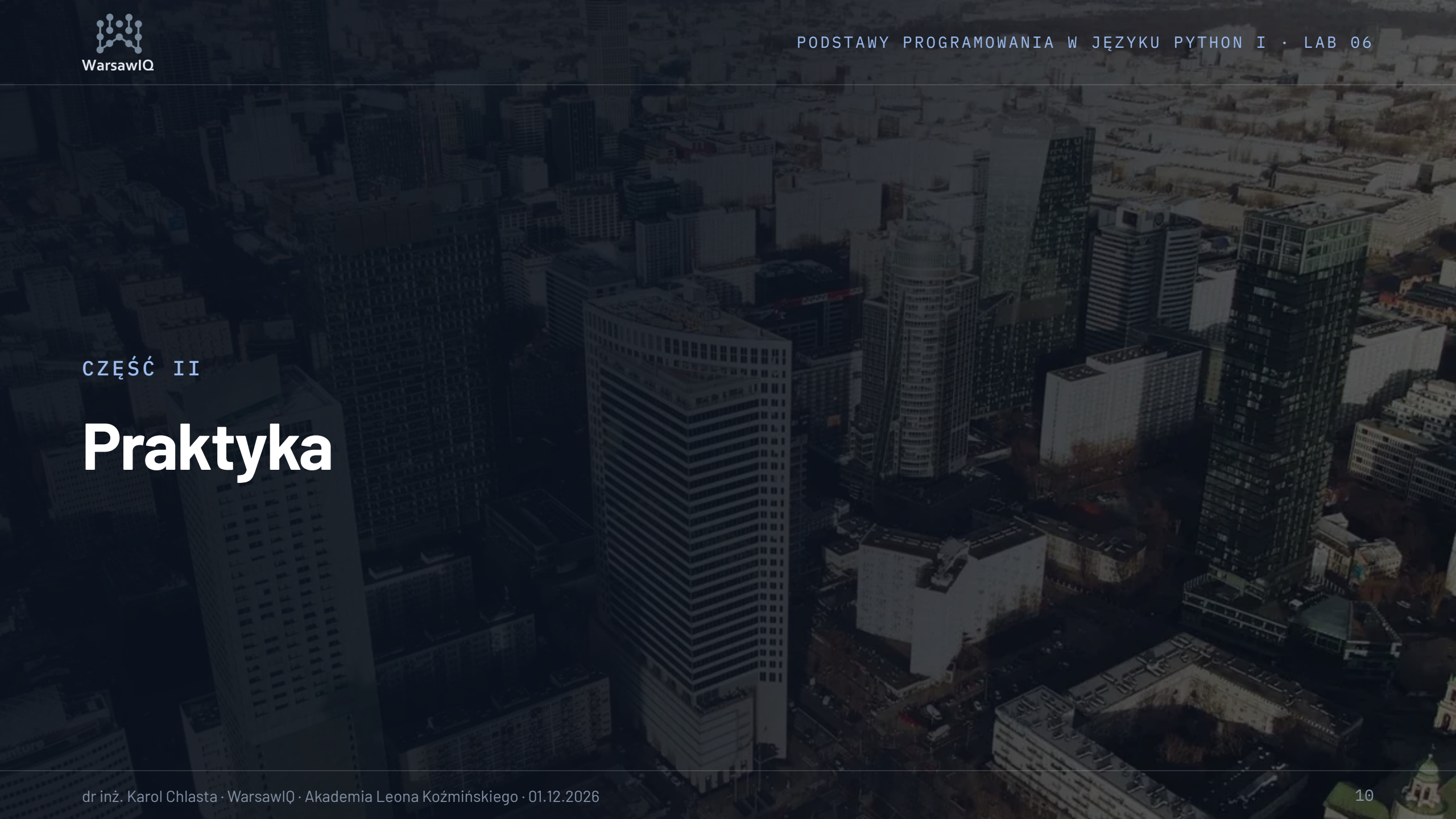
- `get(word, 0)` rozwiązuje problem „pierwszego wystąpienia” bez `if`
- W bibliotece standardowej jest gotowiec: `collections.Counter`

```
from collections import Counter
Counter("to jest to co to jest".split()).most_common(2)
# [('to', 3), ('jest', 2)]
```

Którą strukturę wybrać?

POTRZEBUJĘ…	STRUKTURA
kolejności i dostępu po numerze	`list`
niezmiennego rekordu	`tuple`
szukania po nazwie / identyfikatorze	`dict`
unikalności i szybkiego `in`	`set`
zliczania wystąpień	`dict` / `Counter`

- Dobry wybór struktury danych często ważniejszy niż sprytny algorytm



CZĘŚĆ II

Praktyka

Słownik jako tablica szyfrująca

```
ALPHABET = "abcdefghijklmnopqrstuvwxyz"
encrypt = {}
for i, letter in enumerate(ALPHABET):
    encrypt[letter] = ALPHABET[(i + 3) % 26]
# {'a': 'd', 'b': 'e', ..., 'x': 'a', 'y': 'b', 'z': 'c'}

"".join(encrypt.get(ch, ch) for ch in "ala ma kota")   # 'dod pd nrwd'
```

- `% 26` „zawija” alfabet: po `z` wraca `a`
- Słownik odwrotny (deszyfrowanie) zbudujesz, zamieniając klucze z wartościami

Ciekawostki: kryptografia 🔐

- Według Swetoniusza **Juliusz Cezar** szyfrował listy, przesuwając litery o 3 – dokładnie jak w `zad3_szyfr.py`
- Szyfr podstawieniowy łamie się **analizą częstości liter** – opisał ją arabski uczony **al-Kindi** w IX w. To samo liczysz w `zad4_czestosc.py`
- **Marian Rejewski, Jerzy Różycki i Henryk Zygalski** złamali niemiecką Enigmę w latach 30. XX w.; w 1939 r. przekazali wyniki Francuzom i Brytyjczykom

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_ksiazka_adresowa.py`	książka adresowa
★	`zad2_duplikaty.py`	duplikaty i zbiory
★	`zad3_szyfr.py`	szyfr Cezara na słowniku
★	`zad4_czestosc.py`	częstość słów
☆	`zad5_znajomi.py`	operacje na zbiorach

Sprawdzenie i oddanie

```
cd lab06_slowniki_zbiory
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 06"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- 📝 **Kolokwium I: piątek 08.01.2027, 14:00–17:15, sala A/124** – zakres: laby 01–07

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 6
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 5
- L. Ramalho, *Zaawansowany Python*, Promise 2020 – rozdz. 3 (słowniki i zbiory)
- S. Singh, *Księga szyfrów* – historia kryptografii od Cezara do Enigmy

Następne zajęcia (środa 02.12, 14:00–17:15, sala A/136): funkcje + blok pracy z AI 🤖